PROJECT DOCUMENTATION

Student Performance Analytics System

A Python data-analysis project using NumPy and Pandas

Project Title	Student Performance Analytics System
Student Name	Suhana Thabassum
College	Malla Reddy College of Engineering and Technology (MRCET), Hyderabad
Course	Python with AI (EWB Courses)
Technologies Used	Python, NumPy, Pandas, CSV, Visual Studio Code
Submission Date	15 October 2026

Table of Contents

1. Project Objective . . . 3
2. Technologies Used . . . 3
3. Dataset Description . . . 4
4. Implementation . . . 4
5. Key Features . . . 6
6. Grading and Result Rules . . . 6
7. Output and Screenshots . . . 8
8. Final Outcome . . . 11
9. Challenges and Learning . . . 11
10. GitHub Repository . . . 12
11. How to Run the Project . . . 12
12. Conclusion . . . 13

1. Project Objective

Teachers and departments regularly need to check how a class is performing. Doing this by hand for every student – adding marks, finding averages, deciding grades, and spotting students who need help – takes time and can lead to calculation mistakes.

The **Student Performance Analytics System** was created to solve this problem. It reads student records from a CSV file and automatically calculates total marks, average marks, grades and pass/fail results. It also produces class-level statistics, a subject-wise analysis, a list of top performers and an attendance report. The main objectives are:

- To load and display student data from a CSV file using Pandas.
- To calculate total marks, average marks, grade and result for every student.
- To summarise overall class performance and subject-wise performance.
- To identify the top three students and the best-performing student.
- To identify students whose attendance is below 75%.
- To practise Python functions, loops, conditional statements, NumPy and Pandas in a practical project.

This is a Python data-analysis project. It uses standard statistics and rule-based grading; no machine learning is used.

2. Technologies Used

Technology	Purpose in this project
Python	Main programming language. Used for functions, loops, conditions and program flow. (Python 3.14.2 is shown in the VS Code status bar of the output screenshots.)
NumPy	Used for numerical calculation: `np.mean()` to find the average of the three subject marks and `np.round()` to round it to two decimal places.
Pandas	Used to read the CSV file into a DataFrame, add new columns, filter rows, and calculate statistics such as mean, max, min, count, `nlargest()` and `idxmax()`.
CSV dataset	`students.csv` stores the 20 student records in a simple text format.
Visual Studio Code	Code editor and integrated terminal used to write and run the program.

3. Dataset Description

The dataset file `students.csv` contains **20 student records**. Each row represents one student and each column stores one detail about that student. The marks are out of 100 and attendance is a percentage.

Column	Description	Type
Student_ID	Unique roll/ID number of the student (101 to 120).	Integer
Name	First name of the student.	Text
Department	Department of the student (CSE or AIML).	Text
Python	Marks obtained in the Python subject.	Integer
DBMS	Marks obtained in the DBMS subject.	Integer
Java	Marks obtained in the Java subject.	Integer
Attendance	Attendance percentage of the student.	Integer

The program adds four more columns while it runs: **Total**, **Average**, **Grade** and **Result**. These are calculated columns and are not stored in the CSV file.

4. Implementation

The whole program is written in a single file, `main.py`. The steps below follow the order in which the program runs.

4.1 Loading the dataset with Pandas

The program imports Pandas and NumPy, and then reads the CSV file into a DataFrame named `df`. The file is opened by its name only, so `students.csv` must be in the same folder as `main.py`. The DataFrame is then printed as the first output.

```
import pandas as pd
import numpy as np

df = pd.read_csv("students.csv")
subjects = ["Python", "DBMS", "Java"]
```

4.2 Functions for total, average, grade and result

Four small functions keep the logic organised and reusable:

- `calculate_total(row)` – adds the marks of the three subjects.
- `calculate_average(row)` – uses NumPy to find the mean of the three marks and rounds it to two decimals.
- `calculate_grade(average)` – returns A, B, C, D or F using the grading rules.
- `check_result(row)` – returns "Pass" or "Fail" based on the minimum mark in each subject.

4.3 Loops and conditional statements

Conditional statements and loops are used in several places:

- `calculate_grade()` uses an `if / elif / else` chain to compare the average with 90, 80, 70 and 60.
- `check_result()` uses a `for` loop over the subject list and an `if` condition. If any subject mark is below 40, it returns "Fail" immediately; if the loop finishes without a low mark, it returns "Pass".
- The subject-wise analysis uses a `for` loop to repeat the same average, highest and lowest calculation for Python, DBMS and Java.
- The attendance section uses an `if / else` statement to print either the list of low-attendance students or a message that there are none.

```
def calculate_grade(average):
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    elif average >= 70:
        return "C"
    elif average >= 60:
        return "D"
    else:
        return "F"

def check_result(row):
    for subject in subjects:
        if row[subject] < 40:
            return "Fail"
    return "Pass"
```

4.4 NumPy calculations and Pandas DataFrame operations

NumPy is used inside `calculate_average()` to calculate and round the average. Pandas then applies the functions to the data and adds the results as new columns:

```
df["Total"]   = df.apply(calculate_total, axis=1)
df["Average"] = df.apply(calculate_average, axis=1)
df["Grade"]   = df["Average"].apply(calculate_grade)
df["Result"]  = df.apply(check_result, axis=1)
```

Here `axis=1` tells Pandas to send one row at a time to the function. The Grade column is created by sending each value of the Average column to `calculate_grade()`. Other Pandas operations used in the program are `len()`, `.mean()`, `.max()`, `.min()`, column selection with double brackets, and `.to_string(index=False)` for clean table output.

4.5 Class statistics

The class performance section prints the total number of students, the class average, the highest and lowest average, and the number of students who passed and failed. Pass and fail counts are found by comparing the Result column with "Pass" or "Fail" and summing the matches.

4.6 Subject-wise analysis

A loop goes through the subject list and prints the average, highest marks and lowest marks for each subject. This shows which subject the class found easier or harder.

4.7 Top performers

`df.nlargest(3, "Average")` selects the three students with the highest average. To find the single best student, `df["Average"].idxmax()` gives the row index of the highest average and `df.loc[...]` retrieves that student's record.

4.8 Attendance analysis

The program prints the class average attendance and then filters the DataFrame with `df[df["Attendance"] < 75]` to list every student whose attendance is below 75%. A student with exactly 75% is not included.

5. Key Features

- **Dataset loading and display** – reads students.csv and prints all 20 records.
- **Total and average marks** – calculated for every student across Python, DBMS and Java.
- **Grade calculation** – A to F assigned from the average marks.
- **Pass/fail identification** – a student must score at least 40 in every subject.
- **Class performance statistics** – student count, class average, highest and lowest average, passed and failed counts.
- **Subject-wise analysis** – average, highest and lowest marks for each subject.
- **Top three students** – ranked by average marks.
- **Best-performing student** – the student with the highest average.
- **Attendance analysis** – class average attendance and the list of students below 75%.

6. Grading and Result Rules

Grade rules (based on the average of Python, DBMS and Java marks):

Grade	Condition
A	Average >= 90
B	Average >= 80 and below 90
C	Average >= 70 and below 80
D	Average >= 60 and below 70
F	Average below 60

Result rule: A student passes only if they score at least 40 marks in **every** subject. If the mark in even one subject is below 40, the result is Fail.

Important note: grade and result are decided separately. A student can get grade F (average below 60) and still have the result "Pass" if all three subject marks are 40 or above. In the output, Arjun (53.33), Vikram (55.67) and Surya (57.00) are examples of this.

7. Output and Screenshots

The screenshots below were taken from the integrated terminal in Visual Studio Code after running `python3 main.py`. They show the actual output of the program. The output is long, so it is shown in three parts.

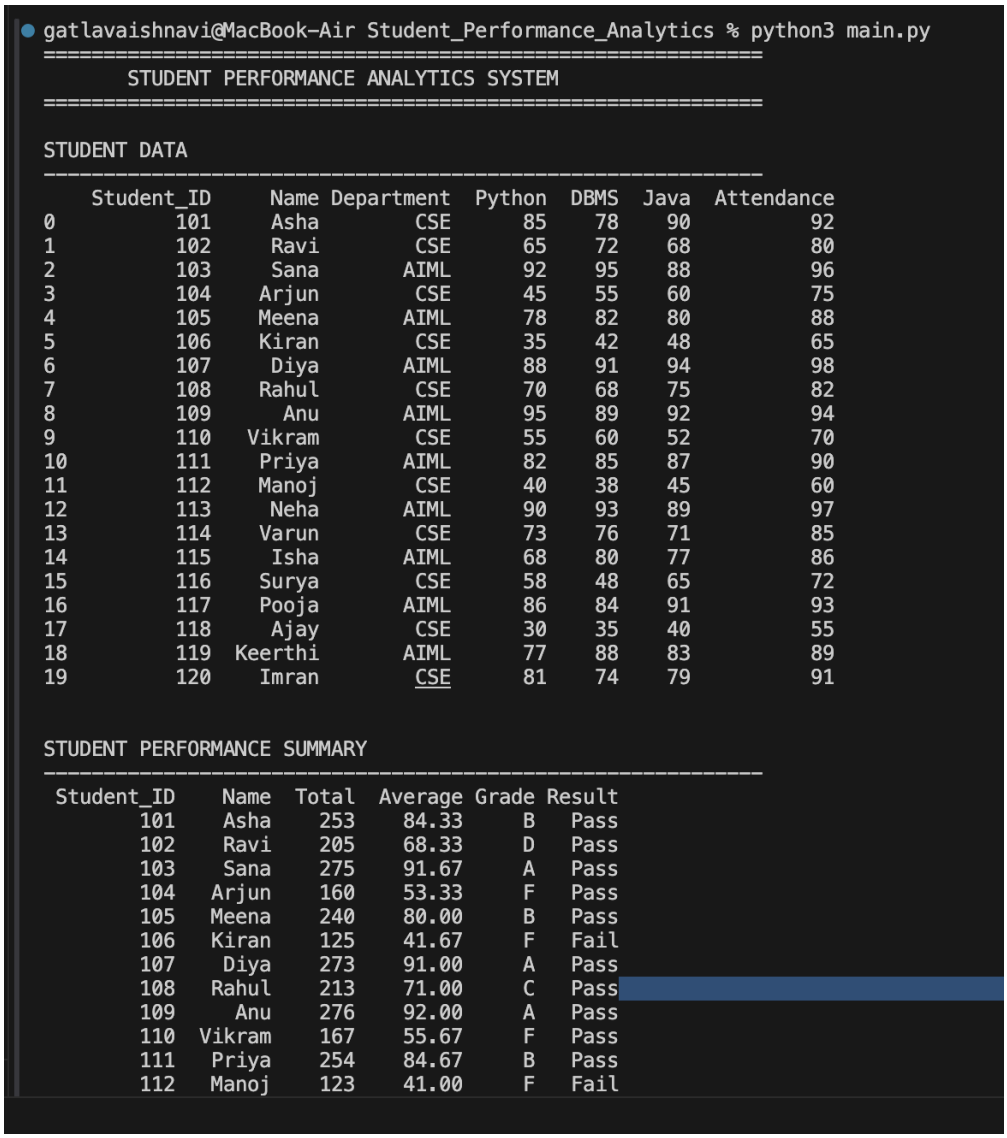


Figure 1: Program title, the student dataset loaded from students.csv (20 records), and the start of the Student Performance Summary showing Total, Average, Grade and Result.

```
        111   Priya    254    84.67     B   Pass
        112   Manoj    123    41.00     F   Fail
        113    Neha    272    90.67     A   Pass
        114   Varun    220    73.33     C   Pass
        115    Isha    225    75.00     C   Pass
        116   Surya    171    57.00     F   Pass
        117   Pooja    261    87.00     B   Pass
        118    Ajay    105    35.00     F   Fail
        119 Keerthi    248    82.67     B   Pass
        120   Imran    234    78.00     C   Pass


CLASS PERFORMANCE
------------------------------------------------------------
Total students: 20
Class average: 71.67
Highest average: 92.0
Lowest average: 35.0
Students passed: 17
Students failed: 3


SUBJECT-WISE PERFORMANCE
------------------------------------------------------------
Python Average: 69.65
Python Highest marks: 95
Python Lowest marks: 30
DBMS Average: 71.65
DBMS Highest marks: 95
DBMS Lowest marks: 35
Java Average: 73.7
Java Highest marks: 94
Java Lowest marks: 40


TOP PERFORMING STUDENTS
------------------------------------------------------------
 Student_ID Name Department  Average Grade
        109  Anu       AIML    92.00     A
        103 Sana       AIML    91.67     A
        107 Diya       AIML    91.00     A

Best performing student: Anu
Highest average marks: 92.0
```

Figure 2: Rest of the performance summary, class performance statistics (20 students, 17 passed, 3 failed), subject-wise analysis, and the top three students with the best-performing student.

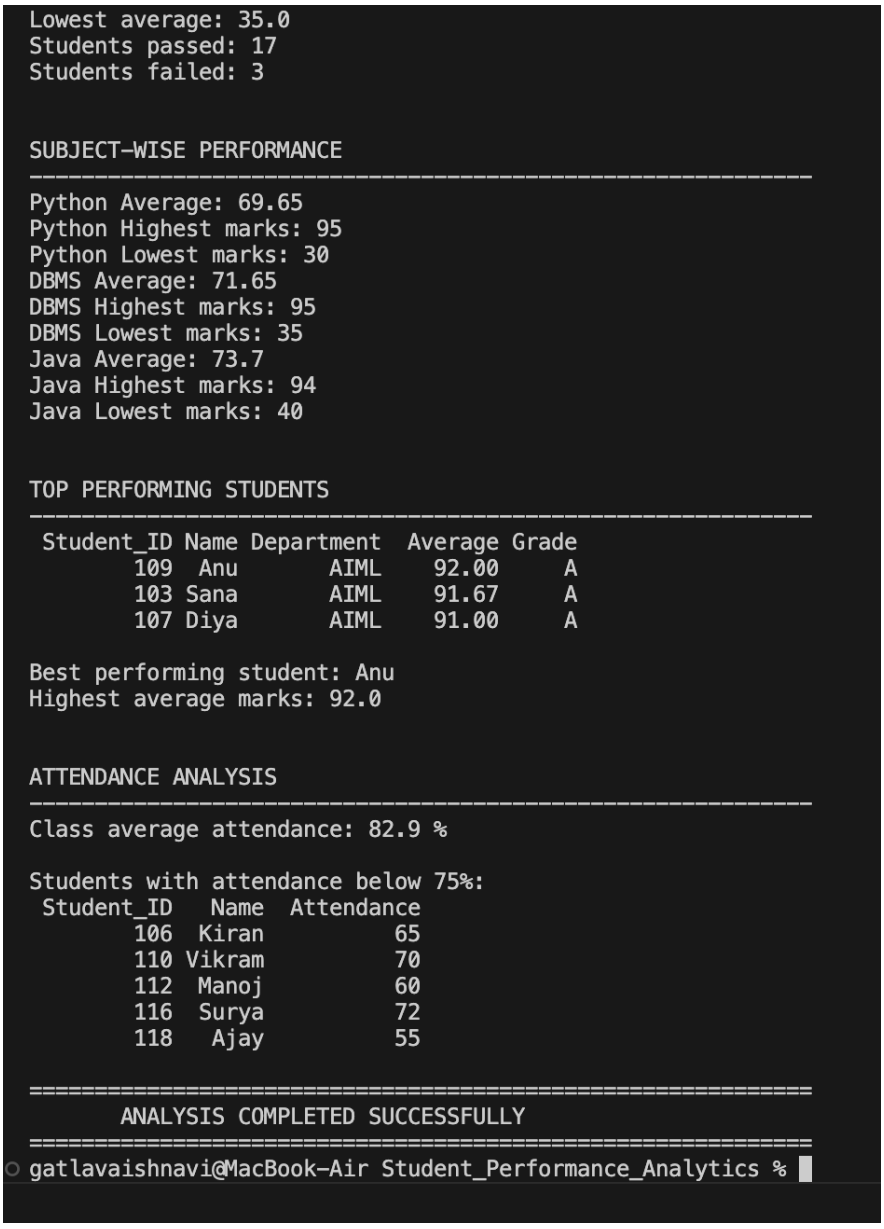


Figure 3: Attendance analysis (class average 82.9%, five students below 75%) and the final "Analysis Completed Successfully" message.

7.1 Summary of the program output

The table below collects the key values printed in the screenshots above.

Item	Output
Total students	20
Class average	71.67
Highest / lowest average	92.0 / 35.0
Students passed / failed	17 / 3 (Kiran, Manoj, Ajay failed)
Python – average / highest / lowest	69.65 / 95 / 30
DBMS – average / highest / lowest	71.65 / 95 / 35
Java – average / highest / lowest	73.7 / 94 / 40
Top three students	Anu (92.00), Sana (91.67), Diya (91.00) – all Grade A, all AIML
Best-performing student	Anu, with an average of 92.0
Class average attendance	82.9 %
Attendance below 75%	Kiran (65), Vikram (70), Manoj (60), Surya (72), Ajay (55)

8. Final Outcome

The completed project reads the student dataset and produces a full performance report in a single run. It successfully:

- loads and displays all 20 student records from a CSV file;
- calculates total marks, average marks, grades and pass/fail results for every student;
- applies the grading rules and the minimum-40-marks rule correctly;
- reports class statistics and subject-wise performance;
- finds the top three students and the best-performing student; and
- identifies the five students with attendance below 75%, who may need attention.

The program runs from the terminal and ends with the message "Analysis Completed Successfully", as shown in Figure 3.

9. Challenges and Learning

Challenges I faced

- **Handling CSV data:** At first I had to understand that the file name in `read_csv()` is relative, so the program only works when students.csv is in the same folder as main.py.
- **Calculating statistics:** I had to choose the correct Pandas methods for the mean, highest and lowest values, and be careful that the class average is the average of the Average column.
- **Applying grading rules:** The order of the `if / elif` conditions matters. I also had to understand that the pass/fail rule depends on each subject, not on the average, which is why a

student can have grade F but still pass.

- **Using Pandas and NumPy together:** It took practice to understand `apply()` with `axis=1` for rows, and to know when to use NumPy (for the calculation) and Pandas (for the table).
- **Formatting the output:** I used `to_string(index=False)` and separator lines to make the terminal output clear and readable.

What I learned

- I learned how to load a dataset with Pandas and add new calculated columns to a DataFrame.
- I learned how to write small, reusable functions and call them on every row of a dataset.
- I learned how loops and conditional statements are used to apply real-world rules such as grading and pass/fail.
- I learned to use NumPy for calculations and Pandas methods such as `nlargest()`, `idxmax()` and boolean filtering.
- I learned how to turn raw data into useful information such as rankings, class statistics and attendance warnings.
- I learned how to organise a project, run it from the terminal and share it on GitHub.

10. GitHub Repository

The complete source code and dataset are available in the following GitHub repository:

https://github.com/suhana-thabassum/Student-Performance-Analytics

The project folder contains the following files:

File	Description
main.py	Main Python program that performs all the analysis.
students.csv	Dataset with 20 student records.
requirements.txt	List of required Python libraries.
README.md	Project description for the repository.

11. How to Run the Project

Step 1 – Install Python. Download and install Python 3 from python.org (or use an existing installation). Check it with `python3 --version`.

Step 2 – Install the dependencies. Open a terminal and run:

```
python3 -m pip install pandas numpy
```

Step 3 – Keep the files together. Place `main.py` and `students.csv` in the same directory, and open a terminal in that directory.

Step 4 – Run the program.

```
python3 main.py
```

The complete analysis report is printed in the terminal.

12. Conclusion

The Student Performance Analytics System shows how Python, NumPy and Pandas can be used to analyse academic data quickly and accurately. By reading a CSV file and applying simple functions, loops and conditions, the program converts raw marks into totals, averages, grades, pass/fail results, class statistics, subject-wise insights, top performers and attendance warnings.

Through this project I strengthened my understanding of Python functions, loops, conditional statements, NumPy calculations and Pandas DataFrame operations. I also learned how to test a program against a dataset, present its output clearly and document a project properly. This project gave me a practical base in data analysis that I can build on in future work.